

CS3490: Homework 2

List recursion and string substitution

1. Objectives

The main objective of this assignment is to reach the mastery with list recursion to the point where you can implement useful functions for manipulating text.

Along the way, you should also strive to achieve good understanding of the following concepts:

1. The basic constructs used in defining functions, including
 - Conditionals: guards and `if-then-else`
 - Making local bindings: `let` and `where`
 - Pattern-matching, and how the type of the input variable determines which patterns are allowed for that variable.
(The possible pattern shapes correspond precisely to the *data constructors* of the input's datatype.)
2. Working with non-recursive and recursive type constructors, and knowing what their data constructors and eliminators are. These include

Type constructor	Syntax	Data constructors	Eliminators
Tuples	<code>(Int, Bool)</code>	<code>(3, True)</code>	<code>fst</code> , <code>snd</code>
Choices	<code>Either Int Bool</code>	<code>Left 3</code> , <code>Right True</code>	<code>either</code>
Options	<code>Maybe Int</code>	<code>Just 3</code> , <code>Nothing</code>	<code>maybe</code>
Lists	<code>[Int]</code>	<code>3 : xs</code> , <code>[]</code>	<code>head</code> , <code>tail</code> , <code>null</code> , <code>fold</code>

3. List recursion. Understanding the logic that goes into coding the recursive case of a list-processing function. (The results of the recursive calls are modified to yield the correct output when list is consed with a new element being put at the head.)
4. How to use list recursion to search and replace substrings.

2. Practice problems

1. Write a function `minList :: [Integer] -> Integer` which returns the minimum element of a list. If the list is empty, you should return 0.

You should have a separate base case for when the list has exactly one element!

2. Write a function `addAbs :: [Integer] -> Integer` which adds together the absolute value of all the elements of a list.

```
addAbs [1,-2,3,-4] = 10
```

```
addAbs [1,10,-100] = 111
```

3. Write a function `existsOdd :: [Integer] -> Bool` which returns `True` if there exists an odd element in the list, and returns `False` otherwise.

4. Write a function `findOdd :: [Integer] -> Maybe Integer` which returns `Just x` if there is some `x` in the input that is odd, and returns `Nothing` otherwise.

5. Write a function `removeEmpty :: [String] -> [String]` which takes a list of strings and removes all empty strings from the list.

```
removeEmpty ["Hello","", "there","", "", "world", ""] = ["Hello", "there", "world"]
```

6. Write a function `subtractEach :: [(Integer,Integer)] -> [Integer]` which takes a list of *pairs* of integers, and outputs a list consisting of their *differences*:

```
subtractEach [(5,1),(6,5),(7,10),(8,15),(9,20)] = [4,1,-3,-7,-11]
```

7. Write a function `makeGreeting :: Maybe String -> String` which takes a string name wrapped in a `Maybe` type, and outputs "Hello, name!".

If no name is provided, it should output "Hello!"

```
makeGreeting (Just "Jesse") = "Hello, Jesse!"
```

```
makeGreeting Nothing = "Hello!"
```

8. Write a function `catMaybes :: [Maybe a] -> [a]` which collects all non-`Nothing` values in the given list and puts them into a list of type `[a]`.

```
catMaybes [Nothing, Just 1, Just 5, Nothing, Nothing, Just 2] = [1,5,2]
```

9. Write a function `classify :: [Either a b] -> ([a],[b])` which takes a list of "Either" type and returns a pair of lists. All `Left` values should go into the first list in the pair, and all `Right` value should go into the second:

```
classify [Right 3, Left "hi", Left "there", Right 10, Right 4] = ([ "hi", "there"], [3,10,4])
```

```
classify [Left 1, Right 'y', Left 5, Left 8, Right 'e', Right 's'] = ([1,5,8], "yes")
```

10. Write a function `isPrefix :: (Eq a) => [a] -> [a] -> Bool` and returns `true` if the first list is a prefix of the second list, and returns `False` otherwise.

```
isPrefix "hey" "hello" = False
```

```
isPrefix [1,2] [1,2,3,4,5] = True
```

3. Homework assignment

Bubblesort

The *bubblesort* algorithm works by exchanging adjacent elements that are out of order until all the elements are in the right order.

1. Write a function `bubble :: Ord a => [a] -> [a]` which recursively goes through the list and interchanges every element that is followed by a smaller element.
`bubble [1,3,2] = [1,2,3]`
`bubble [1,3,5,4,3,2] = [1,3,4,3,2,5]`
2. Write a function `bubbleSort :: Ord a => [a] -> [a]` which repeatedly applies the `bubble` operator to the given list until it has no effect. (So that calling `bubble` results in the same list.)

Generating, searching and replacing strings

If u is a string, then v is a *substring* if u can be written as $u = xvy$ for some strings x and y . The strings x and y can be empty. In particular, `ba` is a substring of `abcba`, as is `abcba`. Note also that the empty string is a substring of every string.

1. Write a function `isSubstring :: String -> String -> Bool` which returns true if the first argument is a substring of the second
`isSubstring "aba" "abcba" = False`
`isSubstring "bc" "abcba" = True`
Hint. Use a helper function `isPrefix` defined in section 2 (the practice set), together with recursive descent through the tail of the list.
2. Write a function `genPrefix :: String -> [String]` which generates all *non-empty* prefixes of the given string and puts them in a list.
`genPrefix "" = []`
`genPrefix "hey" = ["h","he","hey"]`
Hint. First, write a helper function that generates all the *tails* of the given list, so that e.g., `genTails "hey" = ["hey","ey","y"]`. This can be done with straightforward recursion. Then use this function in combination with `reverse` to define `genPrefix`.
3. Write a function `genSubstrings :: String -> [String]` which generates all the substrings of a given string and puts them in a list.
`genSubstrings "abcd" = ["","a","ab","b","abc","bc","c","abcd","bcd","cd","d"]`
You may list the substrings in a different order, but your list should avoid any duplicate entries.
Hint. Use `genPrefix` as a helper function.

4. Write a function `replacePrefix :: (String,String) -> String -> String` which takes a pair `(old,new)` of strings, another string `str`, and replaces the prefix `old` of `str` with the string `new`.

The function can have undefined behavior if `old` is not a prefix of `str`.

```
replacePrefix ("be","spe") "bear" = "spear"
```

Hint. There is a simple solution to this question using the `drop` function.

5. Write a function `replaceString :: (String,String) -> String -> String` which replaces the first occurrence of a substring with a new string:

```
replaceString ("morning","evening") "Good morning!" = "Good evening!"
```

A simple cypher

A *substitution cypher* is a simple encoding technique where each character in the input text is mapped to another character according to some fixed permutation of the alphabet.

We can represent such a permutation by a lookup table — a list of key–value pairs.

1. Write a function `lookUp :: Char -> [(Char,Char)] -> Char` so that calling `lookUp x perm` searches the list `perm` for the pair `(x,y)` whose first coordinate is the given character `x`. It should then output the second coordinate `y`.

```
lookUp 'C' [('A','X'),('B','Y'),('C','Z'),('D','W')] = 'Z'
```

2. Write a function `encode :: [(Char,Char)] -> String -> String` which takes a lookup table and a string and replaces every character by the value it is mapped to by the given table.

```
encode [('A','X'),('B','Y'),('C','Z'),('D','W')] "BABA" = "YXYX"
```

(How could you implement the decode function?)

3. Write a function `makeTable :: String -> String -> [(Char,Char)]` which takes two strings and creates a table by pairing up their characters at the same position.

```
makeTable "ABCD" "XYZW" = [('A','X'),('B','Y'),('C','Z'),('D','W')]
```

Once you implemented the functions above, you can make a Caesar cypher by rotating the English alphabet by a fixed number of letters:

```
caesar :: Int -> [(Char,Char)]
```

```
caesar n = (' ',' ') : makeTable abc (drop n (cycle abc)) where abc = ['A'..'Z']
```

```
ghci> encode (caesar 1) "HOW ARE YOU"
"IPX BSF ZPV"
```

```
ghci> encode (caesar 10) "THIS HOMEWORK IS HARD"
"DRSC RYWOGYBU SC RKBN"
```

```
ghci> encode (caesar 16) it
"THIS HOMEWORK IS HARD"
```